



Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

Oyun Durumu ve Veri Yönetimi

PyGame ile Oyun Geliştirme – Hafta 12

Dr. Tolga BERBER

Bu Hafta Ne Öğreneceğiz?

- 1 State Machine Tasarımı
- 2 Dosya İşlemleri
- 3 Save/Load Sistemi
- 4 Ayarlar ve Konfigürasyon

State Machine Tasarımı

Neden State Machine?

- Oyunun ekranları: Menu, Play, Pause, GameOver, Settings, Load...
- Tek bir main döngüsünde if-elif = kabus
- Her yeni state üç farklı yere kod ekler
- Pause sonrası geri dönüş manuel yönetim ister

Çözüm

Her state = kendi sınıfı
Manager = yığın
Geçişler temiz

Soyut State Sınıfı

```
class State:
    def __init__(self, manager):
        self.manager = manager

    def gir(self): pass           # Yasam dongusu kancalari
    def cik(self): pass

    def olaylari_isle(self, olaylar): pass
    def guncelle(self, dt): pass
    def ciz(self, ekran): pass
```

Üç Klasik Çağrı + İki Kanca

olaylari_isle, guncelle, ciz her state'in standart işlevi. gir/cik müzik, font, snapshot gibi yaşam döngüsü işleri.

StateManager (Yığın Tabanlı)

```
class StateManager:
    def __init__(self): self._yigin = []

    @property
    def aktif(self):
        return self._yigin[-1] if self._yigin else None

    def it(self, state):
        self._yigin.append(state); state.gir()

    def pop(self):
        if self._yigin: self._yigin.pop().cik()

    def degistir(self, state):
        if self._yigin: self.pop()
        self.it(state)
```

Üç Geçiş Türü

it (push)

Yeni state üste eklenir, alttaki yığında *bekler*.
Pause, Dialog, Inventory için ideal.

pop

Üstteki çıkar, alttaki tekrar aktif olur.
Pause'dan oyuna dönüş.

degistir (replace)

Üstteki yok olur, yenisi gelir.
Menu → Play, GameOver → Menu.

PauseState: Snapshot Tekniği

```
class PauseState(State):
    def __init__(self, manager, snapshot=None):
        super().__init__(manager)
        self.snapshot = snapshot

    def gir(self):
        self.altyuz = pygame.Surface((W, H))
        if self.snapshot:
            self.snapshot.ciz(self.altyuz)
        kararti = pygame.Surface((W, H))
        kararti.set_alpha(140)
        kararti.fill((0, 0, 0))
        self.altyuz.blit(kararti, (0, 0))
```

Snapshot Bir Kez

Her karede yeniden çizmek gereksiz; gir'de bir kez yakala, kullan.

Ana Döngü= 18 Satır

```
manager = StateManager()
manager.it(MenuState(manager))

while calistir and manager.aktif:
    dt = saat.tick(60) / 1000.0
    olaylar = pygame.event.get()
    for olay in olaylar:
        if olay.type == pygame.QUIT:
            calistir = False

    manager.aktif.olaylari_isle(olaylar)
    manager.aktif.guncelle(dt)
    manager.aktif.ciz(ekran)
    pygame.display.flip()
```

Yeni state eklemek için ana döngü değişmez!

Dosya İşlemleri

open + with Bloğu

```
# YANLIŞ:
dosya = open("kayit.txt", "w")
dosya.write("...")
# Eger bir hata cikarsa, close cagrilmaz!

# DOGRU --- with bloğu otomatik close:
with open("kayit.txt", "w", encoding="utf-8") as f:
    f.write("Skor: 1500\n")
    f.write("Bolum: 3\n")
# Burada f kapatilmistir (hata olsa bile)
```

Mod karakterleri

"r"=oku, "w"=yaz (siler), "a"=ekle, "x"=yeni, "rb"/"wb"=binary

JSON: Yapılandırılmış Veri

```
import json

# Yazma --- DOGRU desen:
with open(yol, "w", encoding="utf-8") as f:
    json.dump(veri, f,
               indent=2,           # insan dostu
               ensure_ascii=False, # Turkce korunur
               sort_keys=True)     # deterministik

# Okuma:
with open(yol, "r", encoding="utf-8") as f:
    veri = json.load(f)          # bir dict
```

ensure_ascii=False

Olmazsa: "Şükrü" → "\u015e\u00fckr\u00fc" okunmaz.

JSON'ın Tutamadıkları

- **set** → `TypeError`; önce `list` yapın
- **tuple** → sessizce listeye dönüşür (geri okurken liste gelir)
- **datetime, bytes, sınıf instance'ı** → doğrudan desteklenmez
- **Çözüm:** `to_dict` / `from_dict` deseniyle manuel dönüşüm

Saklayabilenler

`dict, list, str, int, float, bool, None`

Atomik Yazma Deseni

```
import os

def atomik_yaz(yol, veri):
    tmp = yol.with_suffix(".json.tmp")
    with open(tmp, "w", encoding="utf-8") as f:
        json.dump(veri, f, indent=2, ensure_ascii=False)
        f.flush()                # Python tampon
        os.fsync(f.fileno())     # OS tampon -> disk

    tmp.replace(yol)              # ATOMIK rename
```

Garanti

Elektrik kesilse bile dosya *ya tam yeni içeriğe sahip, ya hiç değişmemiş.*

Avantajlar:

- Sınıf instance'ı doğrudan
- set, tuple korunur
- Hızlı, küçük dosya

Dezavantajlar:

- Sadece Python
- Binary, insan okumaz
- **KOD YÜRÜTÜR** (güvenlik)
- Sürüm geçişi zor

Bu Kitabın Tercihi: JSON

Save ve settings için JSON. Pickle yalnızca özel performans gereksinimlerinde.

Save/Load Sistemi

Tipik Kayıt Formatı

save_slot1.json

```
{"version": 1, "kayit_zamani": "2026-05-04T14:30:22", "oyuncu_adi": "Ayse",  
"toplam_sure": 1820.5, "oyuncu": {"x": 320, "y": 480, "can": 4, ... },  
"seviye": {"ad": "orman_2", ... }, "ilerleme": {"tamamlanan_seviyeler":  
[...], ... }}
```

Kural

version ilk alan. Yeni özellikler eski kayıtları kırmaz.

to_dict / from_dict Deseni

```
class Oyuncu:
    def to_dict(self):
        return {
            "x": self.x, "y": self.y,
            "can": self.can, "max_can": self.max_can,
            "altin": self.altin,
            "envanter": [e.to_dict() for e in self.envanter],
        }

    @classmethod
    def from_dict(cls, d):
        oyuncu = cls(x=d["x"], y=d["y"],
                     can=d.get("can", 5),
                     altin=d.get("altin", 0))
        oyuncu.max_can = d.get("max_can", oyuncu.can)
        oyuncu.envanter = [Esys.from_dict(x)
                           for x in d.get("envanter", [])]
```

SaveManager Çekirdeği

```
class SaveManager:
    def __init__(self, dizin=None):
        self.dizin = dizin or (Path.home() / ".oyun")
        self.dizin.mkdir(parents=True, exist_ok=True)

    def slot_yolu(self, slot):
        return self.dizin / f"slot{slot}.json"

    def kaydet(self, slot, oyuncu_dict, seviye, ilerleme,
               toplam_sure=0):
        # 1) Yedek (.bak)
        # 2) Atomik (.tmp + replace)
        # 3) version + kayit_zamani metadata
        ...
```

Yükleme + Sürüm Migrasyonu

```
def yukle(self, slot):
    yol = self.slot_yolu(slot)
    if not yol.exists(): return None
    try:
        with open(yol, "r", encoding="utf-8") as f:
            veri = json.load(f)
    except json.JSONDecodeError:
        return self._yedekten_yukle(slot)  # .bak'a dus
    return self._migrate(veri)

def _migrate(self, veri):
    if veri.get("version", 1) < 2:
        veri["oyuncu"].setdefault("max_can", 5)
        veri["version"] = 2
    return veri
```

Slot Organizasyonu

- **Slot 0:** Otomatik kayıt (autosave)
- **Slot 1, 2, 3:** Manuel oyuncu slotları
- Her slotN.json yanında slotN.json.bak

LoadState UI

- Slot listesi: ad, seviye, süre, altın, kayıt zamanı
- OK ile seç, ENTER ile yükle, DEL ile sil
- Boş slot: “– BOS –”
- Bozuk slot: kırmızı “BOZUK”

```
class SaveManager:
    AUTOSAVE_SLOT = 0
    aralik = 60.0      # saniye

    def autosave_kontrol(self, oyun_zamani, **veri):
        if oyun_zamani - self._son_auto >= self.aralik:
            self.kaydet(self.AUTOSAVE_SLOT, **veri)
            self._son_auto = oyun_zamani

# PlayState her karede:
save_mgr.autosave_kontrol(
    pygame.time.get_ticks() / 1000.0,
    oyuncu_dict=self.oyuncu.to_dict(),
    seviye_dict=self.seviye.snapshot(),
    ilerleme_dict=self.ilerleme.to_dict(),
    toplam_sure=self.sure)
```

Ayarlar ve Konfigürasyon

Ayar Kategorileri

Ses

- ses_genel (master)
- ses_muzik
- ses_efekt
- sessiz (toggle)

Kontrol

- tus_sol, tus_sag
- tus_zipla
- tus_pause

Görsel

- tam_ekran
- çözünürlük
- v-sync

Oyun

- zorluk
- dil
- autosave_aralik

Settings Sınıfı

```
DEFAULTS = {  
    "version": 1,  
    "ses_muzik": 0.7, "sessiz": False,  
    "tam_ekran": False,  
    "tus_sol": pygame.K_LEFT, "tus_sag": pygame.K_RIGHT,  
    "zorluk": "orta",  
}
```

```
class Settings:  
    def __init__(self, yol):  
        self.yol = Path(yol)  
        self._veri = dict(DEFAULTS)  
        self.yukle()  
  
    def __getitem__(self, k): return self._veri[k]  
    def __setitem__(self, k, v):  
        self._veri[k] = v; self.kaydet()
```

Yükleme + Birleştirme + Validasyon

```
VALIDATORS = {
    "ses_muzik": lambda v: 0.0 <= v <= 1.0,
    "zorluk":     lambda v: v in ("kolay", "orta", "zor"),
}

def yukle(self):
    if not self.yol.exists(): return
    with open(self.yol, "r", encoding="utf-8") as f:
        kullanici = json.load(f)
    for k, v in kullanici.items():
        if k not in DEFAULTS: continue      # Yutulur
        if k in VALIDATORS and not VALIDATORS[k](v):
            continue                        # Reddedilir
    self._veri[k] = v
```

Ayar Menüsü: 4 Widget Türü

Slider

Ses seviyesi gibi 0–1 arası değer. Sol/sağ ok ile ± 0.05 .

Toggle

Açık/kapalı. ENTER ile değişir.

Seçim

Çoklu seçenek (zorluk, dil). Sol/sağ ok ile cycle.

Tuş Atama

ENTER ile “tuşa basın” modu. Sıradaki KEYDOWN atanır.

Tuş Atama: Çakışma Kontrolü

```
def _tus_atandi(self, tus):  
    if tus == pygame.K_ESCAPE:  
        self.tus_atama_modu = False  
        return # Iptal  
  
    # Cakisma kontrolu  
    for satir in self.satirlar:  
        if satir["tip"] != "tus": continue  
        if (satir["ad"] != mevcut.ad and  
            self.settings[satir["ad"]] == tus):  
            self._uyari("Bu tus zaten atanmis!")  
            self.tus_atama_modu = False  
            return  
  
    self.settings[mevcut.ad] = tus  
    self.tus_atama_modu = False
```

- ① **State Machine:** Soyut State + StateManager + 4 somut state
- ② **Dosya İşlemleri:** open + with, JSON, pickle, atomik yazma
- ③ **Save/Load:** SaveManager, çoklu slot, sürüm migrasyonu, autosave
- ④ **Ayarlar:** Settings sınıfı, üç erişim stili, ayar menüsü, key-binding

Bölüm Sonu Çıktısı

Zıplayan Macera v4: state machine tabanlı, kalıcı kayıtlı, kişiselleştirilebilir oyun.

Bölüm 13: Gelişmiş Konular ve Optimizasyon

- Performans profillemesi
- Veri-odaklı tasarım
- İleri animasyon teknikleri
- Bellek yönetimi

Sorular?